Tutorial de Git - Capítulo 1

1. Introducción al Control de Versiones

- **¿Qué es el Control de Versiones?:** Es un sistema que registra los cambios realizados en un archivo o conjunto de archivos a lo largo del tiempo para poder recuperar versiones específicas más adelante.
- **Ventajas principales:** Permite revertir archivos o proyectos enteros a estados anteriores, comparar cambios a lo largo del tiempo, identificar quién introdujo un problema y recuperar archivos perdidos.
- **Tipos de VCS:**
    - **Locales:** Copian archivos en directorios alternativos o usan bases de datos simples en el equipo local.
    - **Centralizados (CVCS):** Utilizan un único servidor central que contiene todos los archivos versionados.
    - **Distribuidos (DVCS):** Cada cliente clona completamente el repositorio, incluyendo su historial completo, lo que permite trabajar de forma local y colaborar mediante múltiples flujos de trabajo.

2. Breve Historia de Git

- Git fue desarrollado en 2005 por la comunidad del kernel de Linux (liderada por Linus Torvalds) tras la revocación de la licencia gratuita del sistema propietario BitKeeper.
- Sus metas de diseño fueron: extrema velocidad, diseño simple, gran soporte para desarrollo no lineal (miles de ramas paralelas), sistema completamente distribuido y capacidad de manejar proyectos masivos como el kernel de Linux de manera eficiente.

3. Conceptos Fundamentales y Filosofía de Git

- **Instantáneas, no diferencias:** A diferencia de otros sistemas que guardan cambios basados en deltas (diferencias de archivos), Git maneja sus datos como un conjunto de instantáneas (snapshots) del proyecto completo a lo largo del tiempo.
- **Integridad y SHA-1:** Git aplica una suma de comprobación (checksum) a cada objeto antes de almacenarlo mediante un hash SHA-1 de 40 caracteres hexadecimales (0–9 y a–f), haciendo imposible alterar archivos sin que Git lo detecte.

- **Los Tres Estados de los Archivos:**
    - **Committed (Confirmado):** Los datos están almacenados de manera segura en la base de datos local.
    - **Staged (Preparado):** Un archivo modificado se ha marcado en su versión actual para ir en la próxima instantánea.
    - **Modified (Modificado):** Se ha cambiado el archivo pero aún no se ha guardado en la base de datos.
- **Las Tres Secciones de un Proyecto Git:**
    - **Directorio de trabajo (Working Tree):** Copia extraída de una versión del proyecto para su modificación en disco.
    - **Área de preparación (Staging Area / Index):** Archivo que almacena la información de lo que irá en la próxima confirmación.
    - **Directorio de Git (Git Directory / .git):** Almacena la metadata y la base de datos de objetos del proyecto; es lo que se copia al clonar un repositorio.
- **Flujo de Trabajo Básico:**

1. Modificar archivos en el directorio de trabajo.
2. Seleccionar y preparar los cambios en el área de preparación (staging area).
3. Confirmar el commit para guardar permanentemente la instantánea en el directorio de Git.

4. Uso de la Línea de Comandos e Instalación

- **Línea de Comandos:** Es el único entorno donde se pueden ejecutar todos los comandos de Git, garantizando funcionalidad completa en cualquier plataforma.
- **Instalación:**
    - En Linux basado en Fedora: sudo dnf install git.
    - En Linux basado en Debian/Ubuntu: sudo apt-get install git.

5. Configuración Inicial (First-Time Setup)

Git incluye la herramienta git config para personalizar el entorno de trabajo.

- **Establecer la identidad del usuario (Obligatorio):**

git config --global user.name "Tu Nombre"

git config --global user.email "tuemail@ejemplo.com"

La opción --global guarda esta información para que se aplique a todos los proyectos del sistema.

- **Configurar el editor por defecto:**

git config --global core.editor emacs

- **Comprobar la configuración:**

git config --list

6. Obteniendo Ayuda

Para consultar la documentación oficial de cualquier comando desde la terminal:

git help <comando>

git <comando> --help

man git-<comando>